고객을 세그먼테이션하자 [프로젝트] - 유재순

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
LIMIT 10
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	536365	85123A	WHITE HANGING HEART T-LIGH...	6	2010-12-01 08:26:
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:
7	536365	21730	GLASS STAR FROSTED T-LIGHT ...	6	2010-12-01 08:26:
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS total_rows
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

[결과 이미지를 넣어주세요]

행	total_rows
1	541909

데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT
  COUNT(InvoiceNo) AS InvoiceNo_count,
  COUNT(StockCode) AS StockCode_count,
  COUNT(Description) AS Description_count,
  COUNT(Quantity) AS Quantity_count,
  COUNT(InvoiceDate) AS InvoiceDate_count,
  COUNT(UnitPrice) AS UnitPrice_count,
  COUNT(CustomerID) AS CustomerID_count,
  COUNT(Country) AS Country_count
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

[결과 이미지를 넣어주세요]

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo_count	StockCode_count	Description_count	Quantity_count	InvoiceDate_count	UnitPrice_count	CustomerID_count	Country_count
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT 'InvoiceNo' AS column_name, COUNTIF(InvoiceNo IS NULL) / COUNT(*) * 100 AS null_rate
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'StockCode', COUNTIF(StockCode IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'Description', COUNTIF(Description IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'Quantity', COUNTIF(Quantity IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'InvoiceDate', COUNTIF(InvoiceDate IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'UnitPrice', COUNTIF(UnitPrice IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'CustomerID', COUNTIF(CustomerID IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
UNION ALL
SELECT 'Country', COUNTIF(Country IS NULL) / COUNT(*) * 100
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	column_name	null_rate
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.268310731137...
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.92669433428...
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` **의** `Description` **을 추출하는 쿼리문을 작성하기**

```
SELECT Description
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE StockCode = '85123A';
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행

행	Description
1	WHITE HANGING HEART T-LIGH...
2	WHITE HANGING HEART T-LIGH...
3	WHITE HANGING HEART T-LIGH...
4	WHITE HANGING HEART T-LIGH...
5	WHITE HANGING HEART T-LIGH...
6	WHITE HANGING HEART T-LIGH...
7	WHITE HANGING HEART T-LIGH...
8	WHITE HANGING HEART T-LIGH...
9	WHITE HANGING HEART T-LIGH...
10	WHITE HANGING HEART T-LIGH...
11	WHITE HANGING HEART T-LIGH...

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE CustomerID IS NULL;
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 data의 행 135,080개가 삭제되었습니다.

남은 데이터는 541,909 - 135,080 = **406,829개** 행

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT *
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
GROUP BY InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
HAVING COUNT(*) > 1
```

[결과 이미지를 넣어주세요]

4837행

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	571034	23245	SET OF 3 REGENCY CAKE TINS	4	2011-10-13 12:47:00 UTC
2	571034	23239	SET OF 4 KNICK KNACK TINS P...	6	2011-10-13 12:47:00 UTC
3	571034	23494	VINTAGE DOILY DELUXE SEWIN...	3	2011-10-13 12:47:00 UTC
4	538826	22749	FELTCRAFT PRINCESS CHARLO...	1	2010-12-14 12:58:00 UTC
5	577228	23048	SET OF 10 LANTERNS FAIRY LI...	1	2011-11-18 12:07:00 UTC
6	577228	84580	MOUSE TOY WITH PINK T-SHIRT	1	2011-11-18 12:07:00 UTC
7	577228	22144	CHRISTMAS CRAFT LITTLE FRI...	1	2011-11-18 12:07:00 UTC
8	577228	23156	SET OF 5 MINI GROCERY MAGN...	1	2011-11-18 12:07:00 UTC
9	577228	22270	HAPPY EASTER HANGING DEC...	1	2011-11-18 12:07:00 UTC
10	577228	22435	SET OF 9 HEART SHAPED BALL...	1	2011-11-18 12:07:00 UTC
11	539419	48138	DOORMAT UNION FLAG	10	2010-12-17 14:10:00 UTC

페이지당 결과 수: 50 1 – 50 (전체 4837행)

중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data` AS
SELECT DISTINCT *
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 이름이 data인 테이블이 교체되었습니다.

11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 결과 시각화

행	unique_invoice_c...
1	22190

- **고유한 `InvoiceNo` 를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM project-a5c96a0d-47d8-4e1f-8ae.modulabs.data
LIMIT 100
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318
11	541998

- **`InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

```
SELECT *
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

[결과 이미지를 넣어주세요]

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	
5	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	
6	C547388	22645	CERAMIC HEART FAIRY CAKE M...	-12	2011-03-22 16:07:00 UTC	
7	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	
8	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	
9	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	
10	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	
11	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	

페이지당 결과 수: 50 1 – 50 (전체 100행)

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100, 1) AS canceled
_rate
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`;
```

[결과 이미지를 넣어주세요]

전체 401,604개 중 약 2.2%가 취소 건

쿼리 결과

작업 정보 | 결과 | 시각화

행	canceled_rate
1	2.2

`StockCode` 살펴보기

- **고유한 `StockCode`의 개수를 출력하기**

```
SELECT COUNT(DISTINCT StockCode) AS unique_stockcode_count
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 시각화

행	unique_stockcod...
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM project-a5c96a0d-47d8-4e1f-8ae.modulabs.data
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

[결과 이미지를 넣어주세요]

1위는 **85123A** (2,065건): WHITE HANGING HEART T-LIGHT HOLDER

경향성: 상위 제품들의 판매 빈도가 2,065 → 1,894 → 1,659 순으로 점점 줄어드는 롱테일 분포 형태
이상치: POST (8위, 1,196건) - 실제 상품이 아닌 배송비 코드로 보임. 분석 시 제거 필요!

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 :

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

- **`StockCode`의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
```

```
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
)
WHERE number_count <= 1;
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
SELECT ROUND(COUNT(*) / (SELECT COUNT(*) FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`) * 10
0, 2) AS abnormal_rate
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
)
WHERE number_count <= 1;
```

[결과 이미지를 넣어주세요]

이상한 StockCode(POST, M, D 등)를 가진 데이터는 전체의 **0.48%** → 비율은 작지만 분석 시 제거하는 게 좋음

쿼리 결과

작업 정보 | 결과 | 시각화

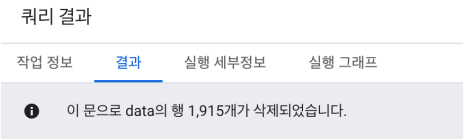
행	abnormal_rate
1	0.48

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
      LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
  )
  WHERE number_count <= 1
);
```

[결과 이미지를 넣어주세요]

비정상 StockCode 행 **1,915개**가 삭제 → 남은 데이터는 401,604 - 1,915 = **399,689개** 행



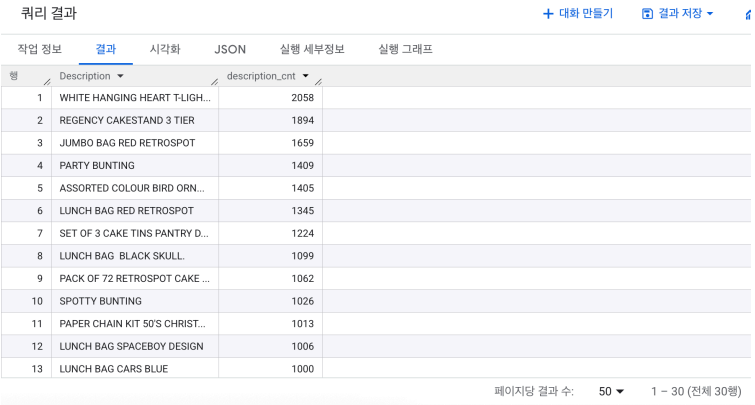
`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

[결과 이미지를 넣어주세요]

LUNCH BAG 시리즈(RED RETROSPOT, BLACK SKULL, SPACEBOY, CARS BLUE)가 여러 개 상위권에 있어요. 런치백이 이 쇼핑몰의 인기 카테고리인 것 같고, 1위는 역시 **WHITE HANGING HEART T-LIGHT HOLDER** (2,058건)!



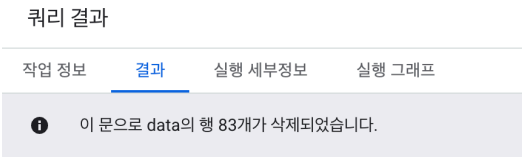
행	Description	description_cnt
1	WHITE HANGING HEART T-LIGH...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORN...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062
10	SPOTTY BUNTING	1026
11	PAPER CHAIN KIT 50'S CHRIST...	1013
12	LUNCH BAG SPACEBOY DESIGN	1006
13	LUNCH BAG CARS BLUE	1000

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE Description IN ('Next Day Carriage', 'High Resolution Image')
```

[결과 이미지를 넣어주세요]

83개 행이 삭제 → 남은 데이터는 399,689 - 83 = **399,606개** 행



- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data` AS
SELECT
  * EXCEPT (Description),
```

```
  UPPER(Description) AS Description
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`;
```

[결과 이미지를 넣어주세요]

- **CREATE OR REPLACE TABLE** → 테이블 교체 성공 ✅
- **DELETE 0개** → 이미 앞에서 'Next Day Carriage', 'High Resolution Image'를 삭제했으니 남은 게 없는 거예요 ✅

상태	종료 시간	SQL	완료된 단계	처리한 바이
✅	PM 12:21 [119:1]	CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data` AS	3	34.7MB
✅	PM 12:21 [125:1]	DELETE	4	34.7MB

`UnitPrice` 살펴보기

- **`UnitPrice`의 최솟값, 최댓값, 평균을 구하기**

```
SELECT # [[YOUR QUERY]] AS min_price, # [[YOUR QUERY]] AS max_price, # [[YOUR QUERY]] AS avg_price
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- **최솟값 0.0** - 가격이 0원인 데이터. 무료 증정품이거나 데이터 오류일 수 있어요
- **최댓값 649.5** - 평균(2.90)에 비해 매우 높은 이상치(outlier)
- **평균**: 약 2.90

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	min_price	max_price	avg_price
1	0.0	649.5	2.904956757405...

- **단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기**

```
SELECT COUNT(*) AS cnt_quantity, MIN(Quantity) AS min_quantity, MAX(Quantity) AS max_quantity, AVG
(Quantity) AS avg_quantity
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE UnitPrice = 0;
```

[결과 이미지를 넣어주세요]

커리큘럼 설명대로 33개는 적은 수이고, Quantity가 정상 범위라 무료 제품보다 데이터 오류일 가능성이 높아 보이네요. 다음 단계에서 이 33개를 삭제필요.

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.5151515151...

- **`UnitPrice = 0`를 제거하고 일관된 데이터셋을 유지하기**

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data` AS
SELECT *
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
WHERE UnitPrice > 0;
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 이름이 data인 테이블이 교체되었습니다.

11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT # [[YOUR QUERY]] AS InvoiceDay, *
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	Cu
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	
3	2010-12-07	537626	84997C	6	2010-12-07 14:57:00 UTC	3.75	
4	2010-12-07	537626	22775	12	2010-12-07 14:57:00 UTC	1.25	
5	2010-12-07	537626	85167B	30	2010-12-07 14:57:00 UTC	1.25	
6	2010-12-07	537626	22725	4	2010-12-07 14:57:00 UTC	3.75	
7	2010-12-07	537626	85232D	3	2010-12-07 14:57:00 UTC	4.95	
8	2010-12-07	537626	20782	6	2010-12-07 14:57:00 UTC	5.49	
9	2010-12-07	537626	22771	12	2010-12-07 14:57:00 UTC	1.25	
10	2010-12-07	537626	22805	12	2010-12-07 14:57:00 UTC	1.25	
11	2010-12-07	537626	22729	4	2010-12-07 14:57:00 UTC	3.75	
12	2010-12-07	537626	22497	4	2010-12-07 14:57:00 UTC	4.25	

페이지당 결과 수: 50 1 – 50 (전체 399573행)

- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT MAX(DATE(InvoiceDate)) AS most_recent_date
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`;
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 시각화

행	most_recent_date
1	2011-12-09

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
```

```
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
GROUP BY CustomerID;
```

[결과 이미지를 넣어주세요]

쿼리 결과 + 대화 만들기 결과 저장

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17
11	12357	2011-11-06
12	12358	2011-12-08
13	12359	2011-12-02

페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
    CustomerID,
    DATE_DIFF(MAX(InvoiceDay) OVER (), InvoiceDay, DAY) AS recency
FROM (
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
    GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

쿼리 결과 + 대화 만들기 결과 저장

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

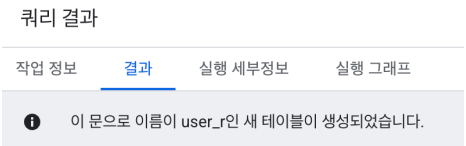
행	CustomerID	recency
1	12346	325
2	12347	2
3	12348	75
4	12349	18
5	12350	310
6	12352	36
7	12353	204
8	12354	232
9	12355	214
10	12356	22
11	12357	33
12	12358	1
13	12359	7

페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_r` AS
SELECT
    CustomerID,
    DATE_DIFF(MAX(InvoiceDay) OVER (), InvoiceDay, DAY) AS recency
FROM (
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
    GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

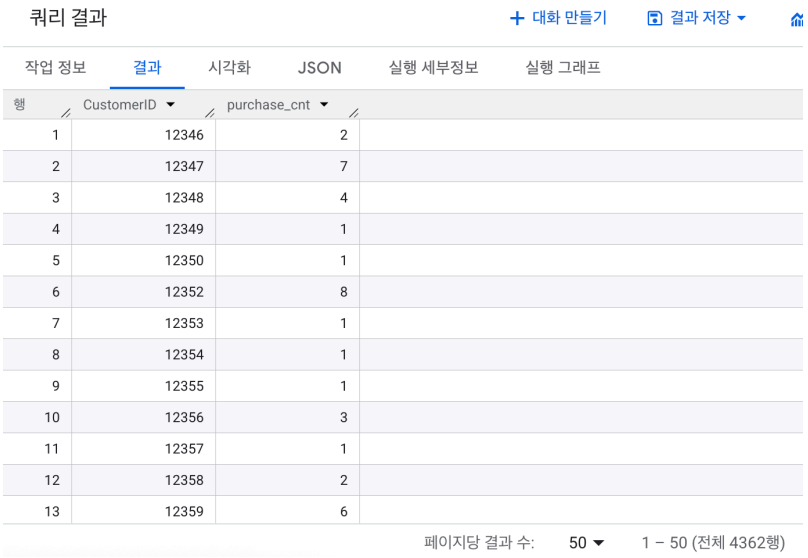


Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
GROUP BY CustomerID;
```

[결과 이미지를 넣어주세요]

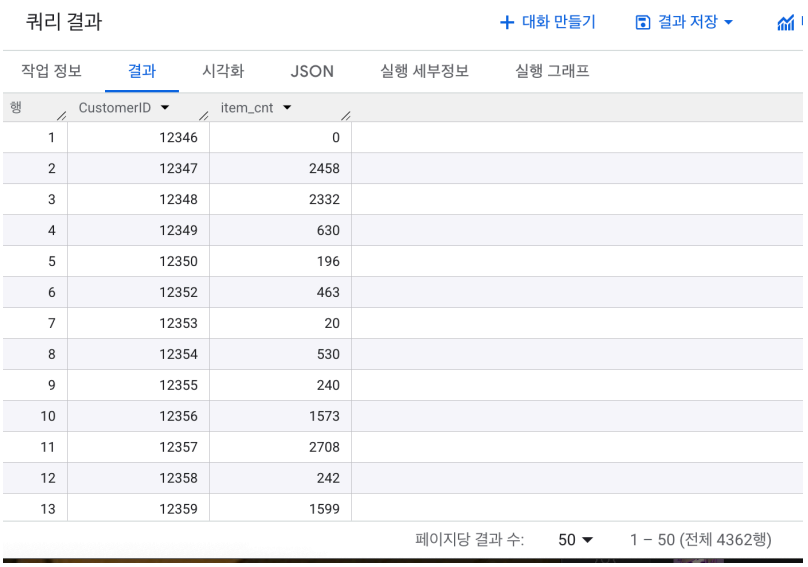


행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1
9	12355	1
10	12356	3
11	12357	1
12	12358	2
13	12359	6

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
GROUP BY CustomerID;
```

[결과 이미지를 넣어주세요]



행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573
11	12357	2708
12	12358	242
13	12359	1599

- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_rf` AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT InvoiceNo) AS purchase_cnt
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
    GROUP BY CustomerID
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
    SELECT
        CustomerID,
        SUM(Quantity) AS item_cnt
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
    GROUP BY CustomerID
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
    pc.CustomerID,
    pc.purchase_cnt,
    ic.item_cnt,
    ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
    ON pc.CustomerID = ic.CustomerID
JOIN `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_r` AS ur
    ON pc.CustomerID = ur.CustomerID;
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보 | 결과 | 실행 세부정보 | 실행 그래프

이 문으로 이름이 user_rf인 새 테이블이 생성되었습니다.

Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
GROUP BY CustomerID;
```

[결과 이미지를 넣어주세요]

쿼리 결과 + 대화 만들기 결과 저장

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.5
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4
11	12357	6207.7
12	12358	928.1
13	12359	6183.0

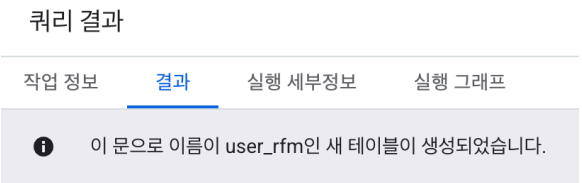
페이지당 결과 수: 50 1 – 50 (전체 4362행)

- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt`로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_rfm` AS
SELECT
    rf.CustomerID AS CustomerID,
    rf.purchase_cnt,
    rf.item_cnt,
    rf.recency,
    ut.user_total,
    ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_average
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_rf` rf
LEFT JOIN (
    -- 고객별 총 지출액
    SELECT
        CustomerID,
        ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
```

```
    GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```

[결과 이미지를 넣어주세요]

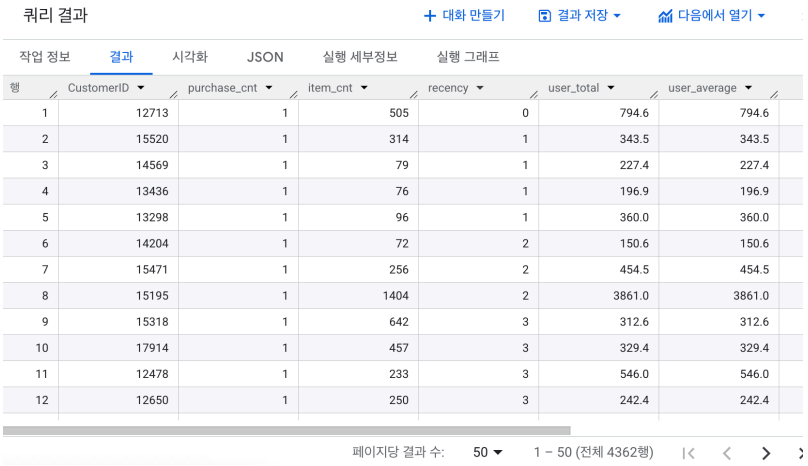


RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_rfm;
```

[결과 이미지를 넣어주세요]



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.6	794.6
2	15520	1	314	1	343.5	343.5
3	14569	1	79	1	227.4	227.4
4	13436	1	76	1	196.9	196.9
5	13298	1	96	1	360.0	360.0
6	14204	1	72	2	150.6	150.6
7	15471	1	256	2	454.5	454.5
8	15195	1	1404	2	3861.0	3861.0
9	15318	1	642	3	312.6	312.6
10	17914	1	457	3	329.4	329.4
11	12478	1	233	3	546.0	546.0
12	12650	1	250	3	242.4	242.4

11-8. 추가 Feature 추출

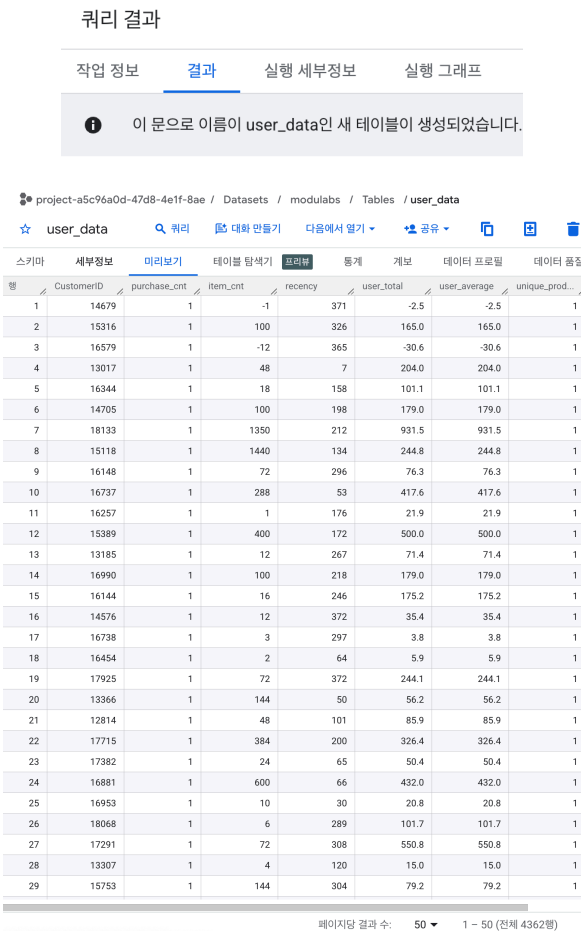
1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_data` AS
WITH unique_products AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT StockCode) AS unique_products
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
    GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_rfm` AS ur
```

```
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]



2. 평균 구매 주기

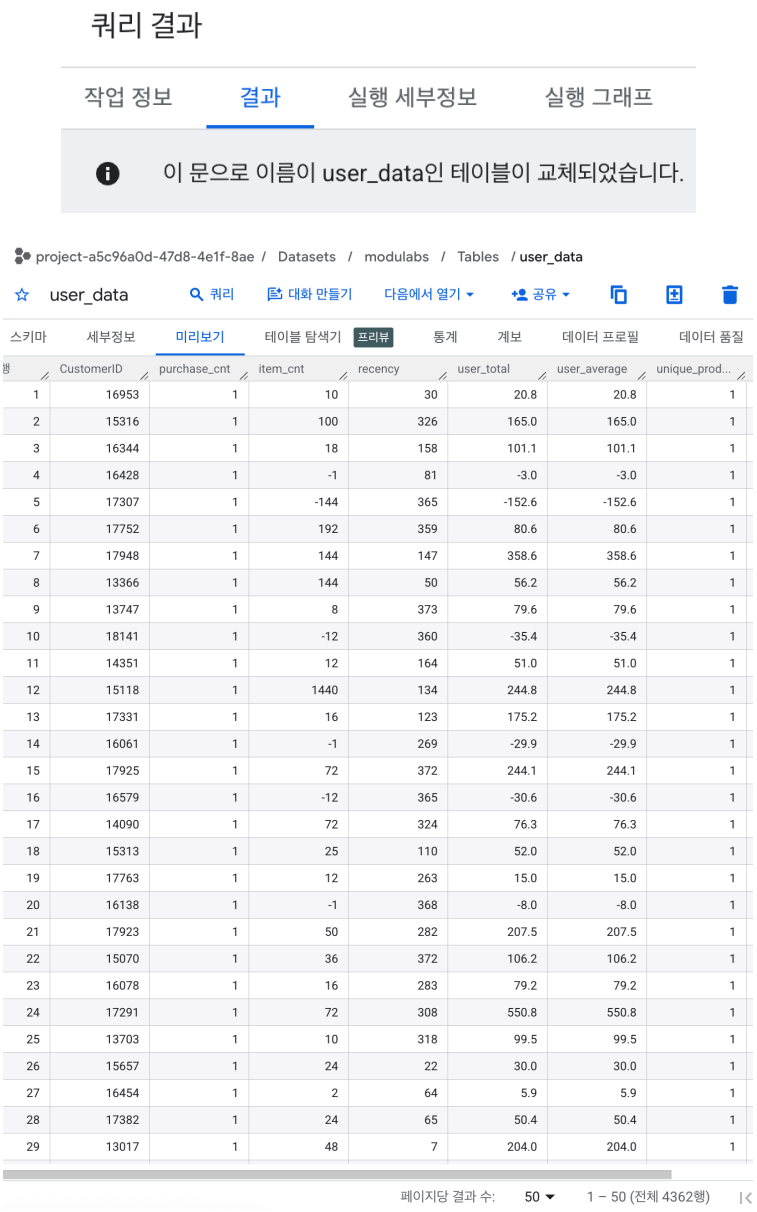
- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data`에 통합**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_in
terval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DA
Y) AS interval_
    FROM
      project_name.modulabs_project.data
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
```

```
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

[결과 이미지를 넣어주세요]



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...
1	16953	1	10	30	20.8	20.8	1
2	15316	1	100	326	165.0	165.0	1
3	16344	1	18	158	101.1	101.1	1
4	16428	1	-1	81	-3.0	-3.0	1
5	17307	1	-144	365	-152.6	-152.6	1
6	17752	1	192	359	80.6	80.6	1
7	17948	1	144	147	358.6	358.6	1
8	13366	1	144	50	56.2	56.2	1
9	13747	1	8	373	79.6	79.6	1
10	18141	1	-12	360	-35.4	-35.4	1
11	14351	1	12	164	51.0	51.0	1
12	15118	1	1440	134	244.8	244.8	1
13	17331	1	16	123	175.2	175.2	1
14	16061	1	-1	269	-29.9	-29.9	1
15	17925	1	72	372	244.1	244.1	1
16	16579	1	-12	365	-30.6	-30.6	1
17	14090	1	72	324	76.3	76.3	1
18	15313	1	25	110	52.0	52.0	1
19	17763	1	12	263	15.0	15.0	1
20	16138	1	-1	368	-8.0	-8.0	1
21	17923	1	50	282	207.5	207.5	1
22	15070	1	36	372	106.2	106.2	1
23	16078	1	16	283	79.2	79.2	1
24	17291	1	72	308	550.8	550.8	1
25	13703	1	10	318	99.5	99.5	1
26	15657	1	24	22	30.0	30.0	1
27	16454	1	2	64	5.9	5.9	1
28	17382	1	24	65	50.4	50.4	1
29	13017	1	48	7	204.0	204.0	1

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    (취소 비율은 소수점 두번째 자리)

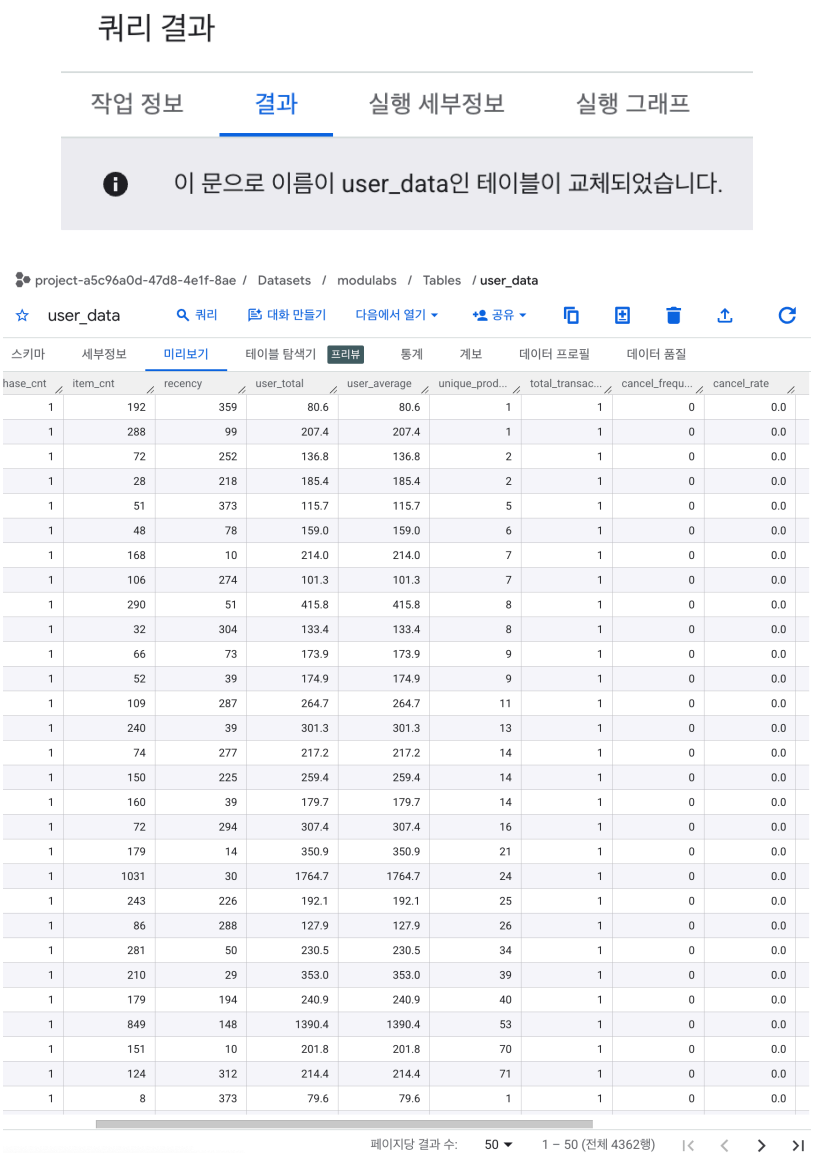
```
CREATE OR REPLACE TABLE `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_data` AS

WITH TransactionInfo AS (
    SELECT
```

```
        CustomerID,
        COUNT(DISTINCT InvoiceNo) AS total_transactions,
        COUNTIF(InvoiceNo LIKE 'C%') AS cancel_frequency
    FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.data`
    GROUP BY CustomerID
)

SELECT u.*, t.* EXCEPT(CustomerID), ROUND(t.cancel_frequency / t.total_transactions * 100, 2) AS canc
el_rate
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;
```

[결과 이미지를 넣어주세요]

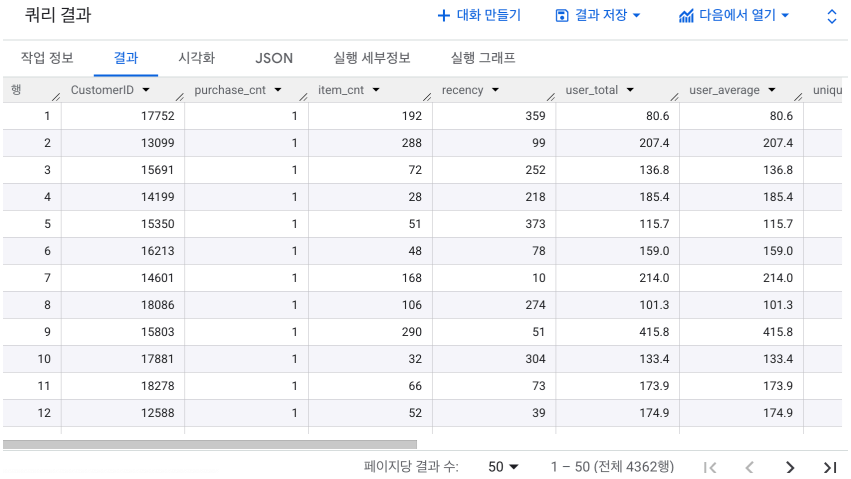


hase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	total_transac...	cancel_frequ...	cancel_rate
1	192	359	80.6	80.6	1	1	0	0.0
1	288	99	207.4	207.4	1	1	0	0.0
1	72	252	136.8	136.8	2	1	0	0.0
1	28	218	185.4	185.4	2	1	0	0.0
1	51	373	115.7	115.7	5	1	0	0.0
1	48	78	159.0	159.0	6	1	0	0.0
1	168	10	214.0	214.0	7	1	0	0.0
1	106	274	101.3	101.3	7	1	0	0.0
1	290	51	415.8	415.8	8	1	0	0.0
1	32	304	133.4	133.4	8	1	0	0.0
1	66	73	173.9	173.9	9	1	0	0.0
1	52	39	174.9	174.9	9	1	0	0.0
1	109	287	264.7	264.7	11	1	0	0.0
1	240	39	301.3	301.3	13	1	0	0.0
1	74	277	217.2	217.2	14	1	0	0.0
1	150	225	259.4	259.4	14	1	0	0.0
1	160	39	179.7	179.7	14	1	0	0.0
1	72	294	307.4	307.4	16	1	0	0.0
1	179	14	350.9	350.9	21	1	0	0.0
1	1031	30	1764.7	1764.7	24	1	0	0.0
1	243	226	192.1	192.1	25	1	0	0.0
1	86	288	127.9	127.9	26	1	0	0.0
1	281	50	230.5	230.5	34	1	0	0.0
1	210	29	353.0	353.0	39	1	0	0.0
1	179	194	240.9	240.9	40	1	0	0.0
1	849	148	1390.4	1390.4	53	1	0	0.0
1	151	10	201.8	201.8	70	1	0	0.0
1	124	312	214.4	214.4	71	1	0	0.0
1	8	373	79.6	79.6	1	1	0	0.0

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM `project-a5c96a0d-47d8-4e1f-8ae.modulabs.user_data`;
```

[결과 이미지를 넣어주세요]



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	uniqu
1	17752	1	192	359	80.6	80.6	
2	13099	1	288	99	207.4	207.4	
3	15691	1	72	252	136.8	136.8	
4	14199	1	28	218	185.4	185.4	
5	15350	1	51	373	115.7	115.7	
6	16213	1	48	78	159.0	159.0	
7	14601	1	168	10	214.0	214.0	
8	18086	1	106	274	101.3	101.3	
9	15803	1	290	51	415.8	415.8	
10	17881	1	32	304	133.4	133.4	
11	18278	1	66	73	173.9	173.9	
12	12588	1	52	39	174.9	174.9	

회고

[회고 내용을 작성해주세요]

Keep : 매우 어렵다.

Problem : 혼자서는 가능하지 않고, gpt의 도움으로 해결했다.

Try : 코드 암기보다 이해를 하는 방향으로 하고 있다.